Inheritance

CCS1200
Introduction to Object-Orientation

Dr Ioanna Stamatopoulou

Inheritance Examples

- A Bedroom **is a kind of** Room
- A ConferenceRoom **is a kind of** Room
- A DiningRoom **is a kind of** Room

- A Dog **is a kind of** Mammal
- A Mammal **is a kind of** Animal

Inheritance

- Inheritance is a relation between classes (**NOT** objects)
- When a class inherits another class it inherits:
  - **all** the attributes
  - **all** the methods

- The principle that knowledge of a more general category is applicable to the more specific category is called **inheritance**

Inheritance (cont'd)

- Mammals nurse their youngs
- Dogs are Mammals so they **inherit** the behavior of nursing their youngs
- Loulou, as an instance of a Dog, will also nurse her young

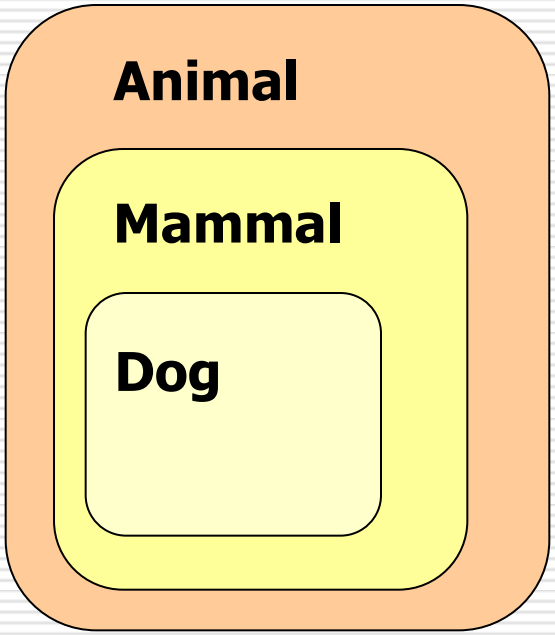
- Animals move spontaneously and independently
- Mammals are Animals, so they move spontaneously and independently (via inheritance)
- So, Dogs, and Loulou as a dog, also move spontaneously and independently

Class Hierarchy

- Loulou is a **Dog**
  - She is also a **Mammal**
- **Mammal** is a class more general than **Dog**

  or…
- **Dog** is a specialized form of **Mammal** .

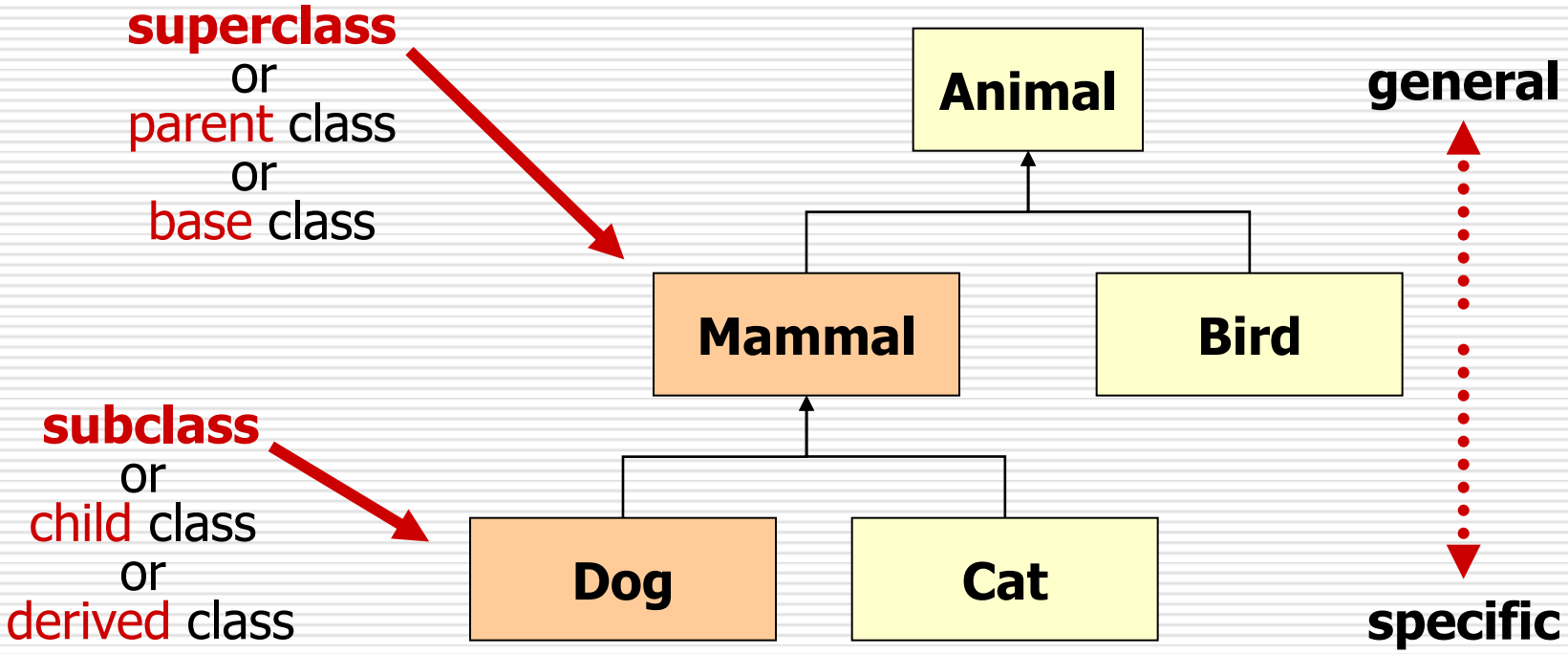
  or…
- **Dog** **is a kind of** **Mammal**



- A **Mammal** is an **Animal**
- **Mammal** is **a kind of** **Animal**

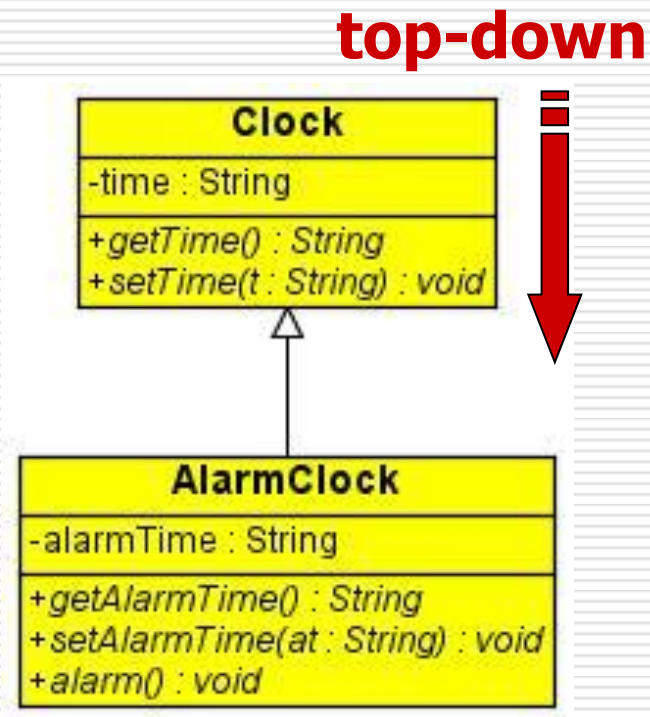
Class Hierarchy in UML

- A Class Hierarchy (or inheritance hierarchy) in UML is illustrated with a tree-like structure



Purpose of inheritance: Specialisation

- Extending the functionality of an existing class
- Example:
  - Existing class: **Clock**
  - Specialised class: **AlarmClock**
  - The class AlarmClock has more attributes and methods than the Clock

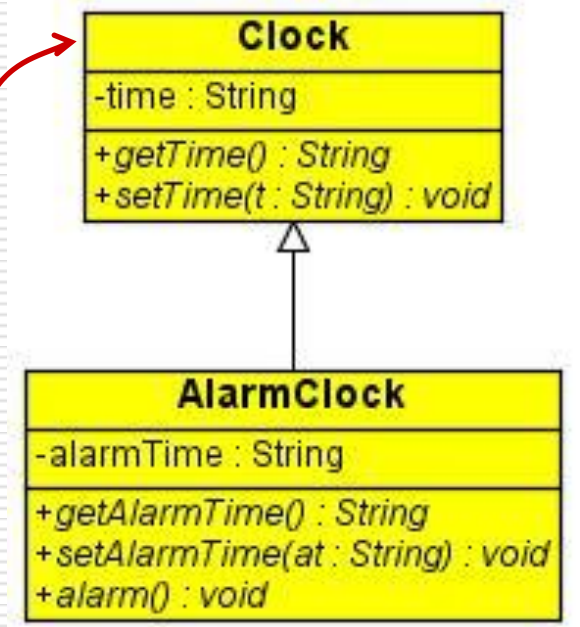


Specialisation

- You need a class AlarmClock with the following behaviour:
  - setTime
  - getTime
  - setAlarmTime
  - getAlarmTime
  - alarm
- You search in your existing classes and find a class Clock:
- You create the class AlarmClock which
  - **inherits** Clock
  - adds extra attributes
  - adds extra methods

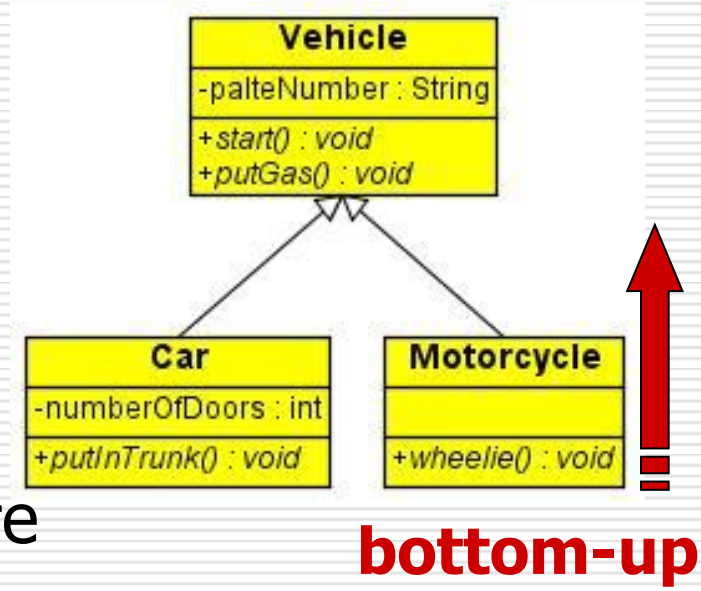
  to model the extended functionality of an alarm clock

Result:
CLASS REUSE!



Purpose of inheritance: Generalisation

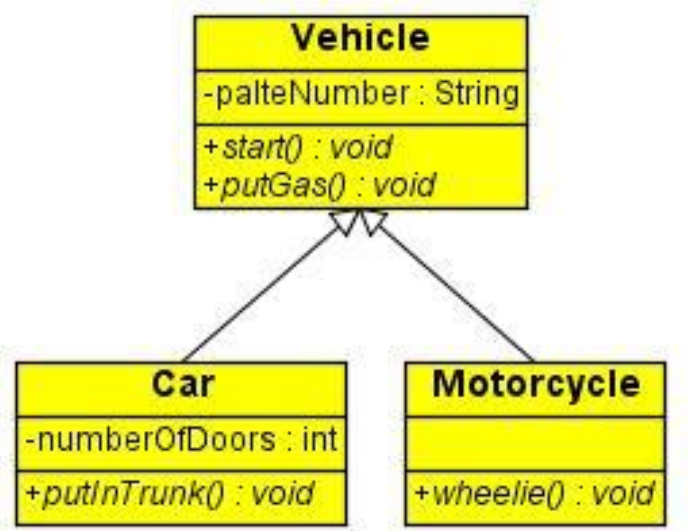
- Sharing common attributes and methods between two or more classes
- Example:
  - Classes **Motorcycle** and **Car** share common attributes and behaviour (methods)
  - Common characteristics are moved to a general class **Vehicle**



Generalisation

- During the problem analysis you identify that you need two classes with the following behaviour:
  - Car
    - **start**
    - **putGas**
    - putInTrunk
  - Motorcycle
    - **start**
    - **putGas**
    - wheelie
- The classes share common behaviour
- You decide to create a general class Vehicle modelling the common behaviour and let Car and Motorcycle inherit Vehicle

Result:
DUPLICATION AVOIDANCE!



Implementing Inheritance

- class Derived inherits class Base
- Derived inherits from Base:
  - the attribute x
  - the methods getX, setX
- Derived extends Base with:
  - the attribute y
  - the methods getY, setY
- Objects of class Base can use:
  - setX and getX
- Objects of class Derived can use:
  - setX, getX (of inherited class)
  - setY, getY

```
class Base  {
  private int x;

  public int getX () { return x; }
  public void setX (int n)  { x = n; }
}

class Derived  extends Base  {
  private int y;

  public int getY () { return y; }
  public void setY (int n) { y = n; }
}

public static void main(String[] args){

  Base b = new Base();
  b.setX(3);
  System.out.println(b.getX());

  Derived d = new Derived();
  d.setX(5);
  System.out.println(d.getX());
  d.setY(5);
  System.out.println(d.getY());
}
```

Accessing inherited attributes

- Attribute **x** has **private** access modifier in class Base
- Although it is inherited, it is **not directly accessible** in class Derived
- So this will cause a compiler error

```
class Base  {
  private int x;
  public int getX () { return x; }
  public void setX (int n)  { x = n; }
}
class Derived extends Base {
  private int y;
  public int getY ()  { return y; }
  public void setY (int n)  { y = n; }
  public boolean isXEqualtoY() {
    return x == y ;
    //access to x is NOT ALLOWED
    //in Derived class
    //if x is private in Base class
  }
}
public static void main(String[] args) {
  Derived d = new Derived;
  d.setX(5);
  d.setY(5);

  if ( d.isXEqualtoY() )
    System.out.print("x and y are equal");
}
```

Accessing inherited attributes

GOOD PRACTICE

- In order to access the inherited attribute x, use the **`getX()`** and **`setX()`** methods (if they are available)

- So this is now correct

```
class Base  {
  private int x;
  public int getX () { return x; }
  public void setX (int n)  { x = n; }
}
class Derived extends Base {
  private int y;
  public int getY ()  { return y; }
  public void setY (int n)  { y = n; }
  public boolean isXEqualtoY() {
    return getX() == y ;
    //use the getX() method to access
    //the value of x in the Derived class
    //if x is private in the Base class
  }
}
public static void main(String[] args) {
  Derived d = new Derived;
  d.setX(5);
  d.setY(5);

  if ( d.isXEqualtoY() )
    System.out.print("x and y are equal");
}
```

Accessing inherited attributes

BAD PRACTICE

- An alternative is to declare the attribute x as **protected** in the Base class
- Protected attributes are directly accessible by:
  - All derived classes
  - **but also to all classes belonging to the same package** and as a result should be avoided when possible

```
class Base  {
  protected int x;
  public int getX () { return x; }
  public void setX (int n)  { x = n; }
}
class Derived extends Base {
  private int y;
  public int getY ()  { return y; }
  public void setY (int n)  { y = n; }
  public boolean isXEqualtoY() {
    return x == y ;
    //access to x ALLOWED in Derived class
    //if x is protected in Base class
  }
}

public static void main(String[] args) {
  Derived d = new Derived;
  d.setX(5);
  d.setY(5);

  if ( d.isXEqualtoY() )
    System.out.print("x and y are equal");
}
```

Calling inherited methods

- Inside a subclass you may call a method that is inherited using the **super** keyword

```
class Base  {
  private  int x;
  public int getX () { return x; }
  public void setX (int n) { x = n; }

  public String toString() {
    return "X is:\t" + x;
  }
}

class Derived extends Base {
  private int y;
  public int getY () { return y; }
  public void setY (int n) { y = n; }
  public boolean isXEqualtoY() {
    return x == y ;
  }
  public String toString() {
    return super.toString() +"\nY is: "+y;
}
```

The **super** keyword

- One use of the super keyword is for calling an inherited method (see example on previous slide) by writing: **`super.inheritedMethodName()`**

- The super keyword may be used on its own inside the constructor of a subclass to call the constructor of the superclass

```
class Base  {
  private int x;
  Base (int xValue) {
    x = xValue;
  }
}

class Derived extends Base {
  private int y;

  Derived (int xValue, int yValue) {
    super(xValue);
    y = yValue;
  }
}
```